

C2_Week2

▼ TensorFlow implementation:

Train a Neural Network in TensorFlow

Given set of (x, y) examples
How to build and train this in code?

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(units=25, activation='sigmoid')
    Dense(units=15, activation='sigmoid')
    Dense(units=1, activation='sigmoid')
])

from tensorflow.keras.losses import BinaryCrossentropy

model.compile(loss=BinaryCrossentropy())

model.fit(X, Y, epochs=100)
```

①
②
③
epochs: number of steps in gradient descent

- Thay vì như tuần trước mình truyền tên của các layer trong model Sequential thì giờ có thể định nghĩa layer trực tiếp vào trong Sequential([...]) luôn.
- Ở đây khi mình compile thì mình phải nêu rõ ra hàm loss và mình sử dụng, trong trường hợp này là BinaryCrossentropy (cho bài toán phân loại.)
- Trường hợp tổng quát:

$$BCE = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

- Trường hợp của một điểm dữ liệu duy nhất:

$$\text{Loss} = -[y \log(p) + (1 - y) \log(1 - p)]$$

▼ Training Details

Model training steps:

1. Tạo Model:

- Định nghĩa model, trong bước này mình sẽ định nghĩa các mạng, và định nghĩa các activation cho từng lớp:

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(units=25, activation='sigmoid')
    Dense(units=15, activation='sigmoid')
    Dense(units=1, activation='sigmoid')
])
```

→ Mô hình có 3 lớp với từng số units tương ứng, và các activation đều là sigmoid.

2. Định nghĩa các hàm mất mát

2. Loss and cost functions

Mnist digit classification problem → binary classification

$$J(W, B) = \frac{1}{m} \sum_{i=1}^m L(f(\vec{x}^{(i)}), y^{(i)})$$

$L(f(\vec{x}), y) = -y \log(f(\vec{x})) - (1 - y) \log(1 - f(\vec{x}))$

Compare prediction vs. target

logistic loss
also known as binary cross entropy

$w^{[1]}, w^{[2]}, w^{[3]}$ $\vec{b}^{[1]}, \vec{b}^{[2]}, \vec{b}^{[3]}$ $f_{W,B}(\vec{x})$

model.compile(loss= BinaryCrossentropy()) from tensorflow.keras.losses import BinaryCrossentropy **K** Keras

regression (predicting numbers and not categories) mean squared error

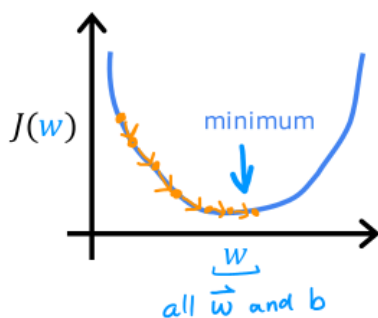
model.compile(loss= MeanSquaredError()) from tensorflow.keras.losses import MeanSquaredError

```
# Cho bài toán phân loại nhị phân (binary classification)
from tensorflow.keras.losses import BinaryCrossentropy
model.compile(loss=BinaryCrossentropy())

# Cho bài toán hồi quy (regression)
from tensorflow.keras.losses import MeanSquaredError
model.compile(loss=MeanSquaredError())
```

3. Gradient Descent

3. Gradient descent



```
repeat {
     $w_j^{[l]} = w_j^{[l]} - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$ 
     $b_j^{[l]} = b_j^{[l]} - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$ 
} Compute derivatives
for gradient descent
using "back propagation"
```

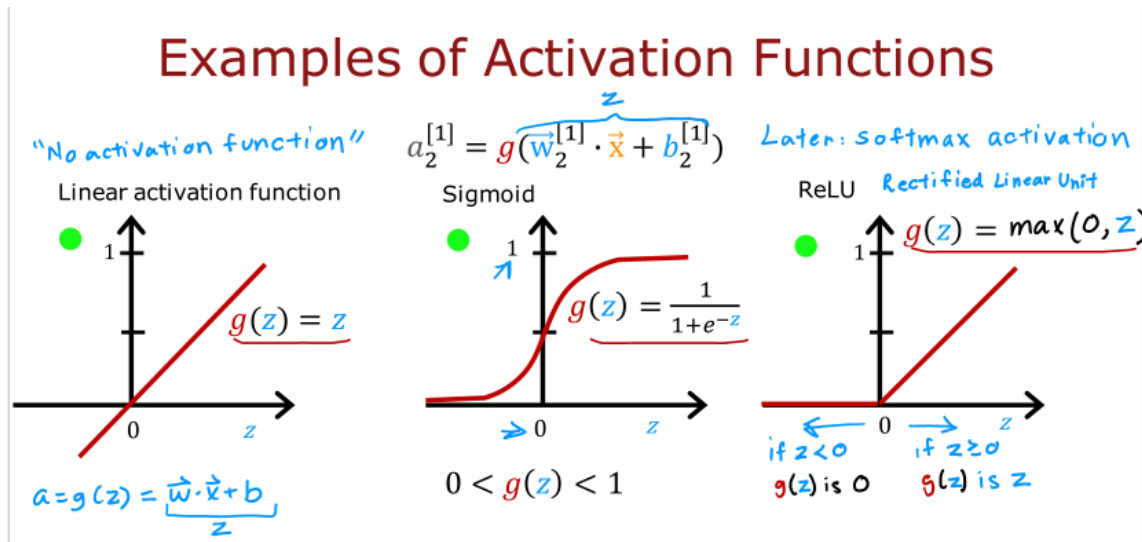
```
model.fit(X, y, epochs=100)
```

```
repeat {
     $w_j^{[l]} = w_j^{[l]} - \alpha \frac{\partial}{\partial w_j^{[l]}} J(\vec{w}, b)$ 
     $b_j^{[l]} = b_j^{[l]} - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$ 
}
```

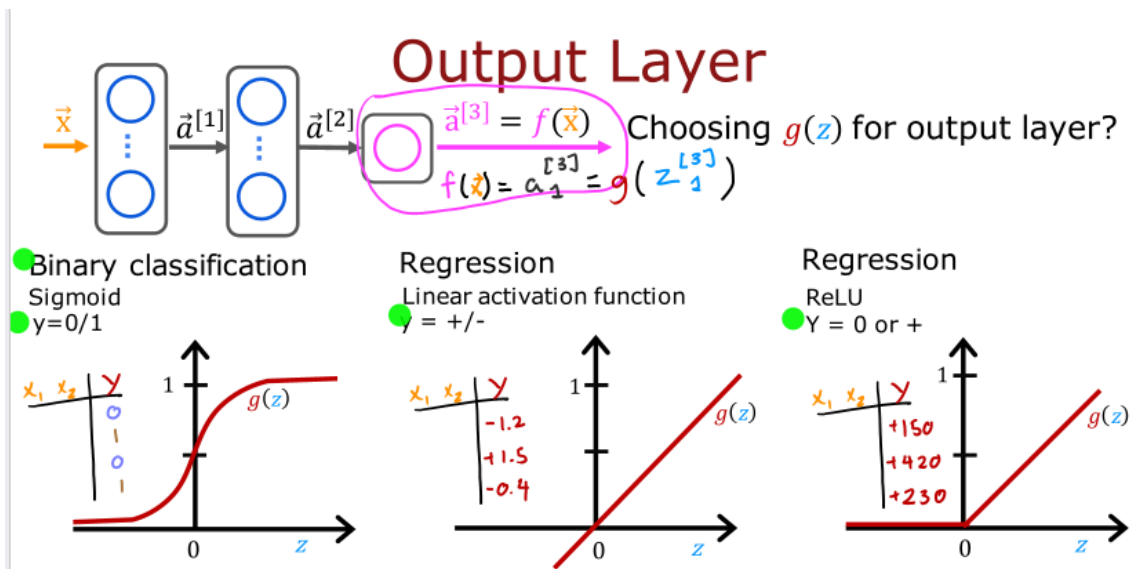
- Nó sẽ tính các đạo hàm bằng cách sử dụng "back propagation"

▼ Alternatives to the sigmoid activation

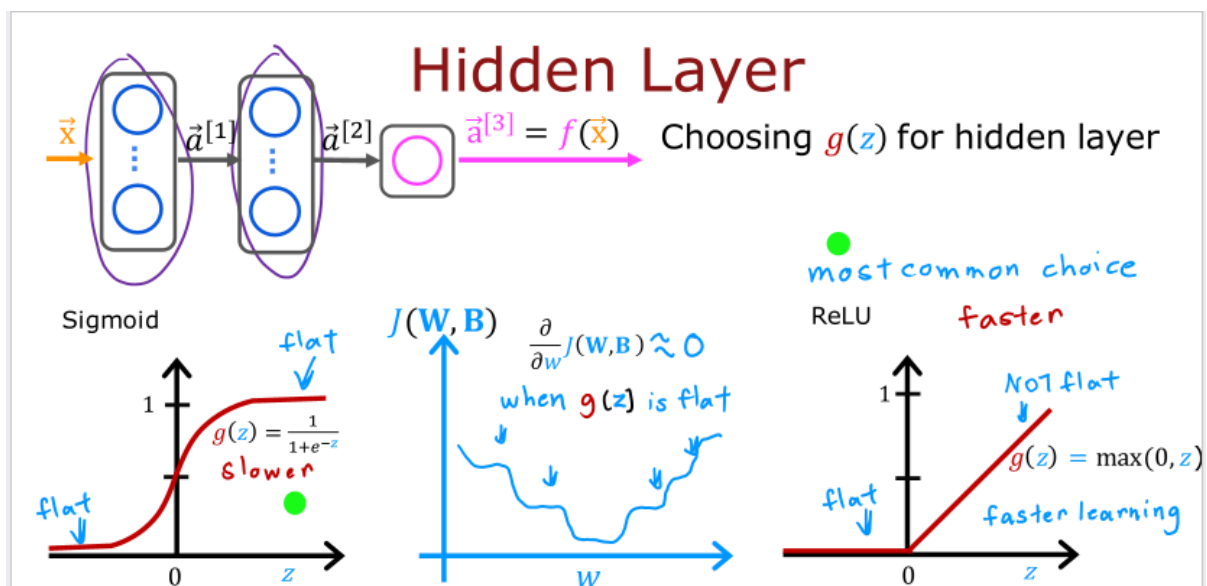
- Được cái về hàm sigmoid là nó tính toán khá là khó, bởi vì nó phải tính phân số của một lũy thừa. Và mặt khác, hàm sigmoid chỉ có thể đưa ra một dự đoán về tỷ lệ phần trăm của output giữa 0 và 1. Không phải bài toán nào cũng rõ ràng về các output 0 và 1, sẽ có những bài toán yêu cầu sự chỉ ra mối quan hệ tương đồng không chỉ nằm lại ở số 0 và 1, mà còn cao hơn thế.



- ReLU được sử dụng nhiều nhất do nó đơn giản chỉ tính $g(z) = \max(0, z)$, dễ tính toán, và nếu một nơ ron nào đó có giá trị âm, nó sẽ được đưa về 0 và coi như không được sử dụng.
- ReLU được sử dụng nhiều nhất.
- Tất nhiên là cái hàm activation cho nơ ron của lớp output vẫn sẽ phụ thuộc vào đề bài:



- Tuy vậy, việc chọn ra hàm activation cho các nơ ron trong các lớp ẩn có thể tăng tốc độ học, trong đó họ sử dụng ReLU nhiều nhất.



1. Binary classification: sigmoid
2. Regression (y neg/ y pos): linear
3. Regression ($y \geq 0$): relu

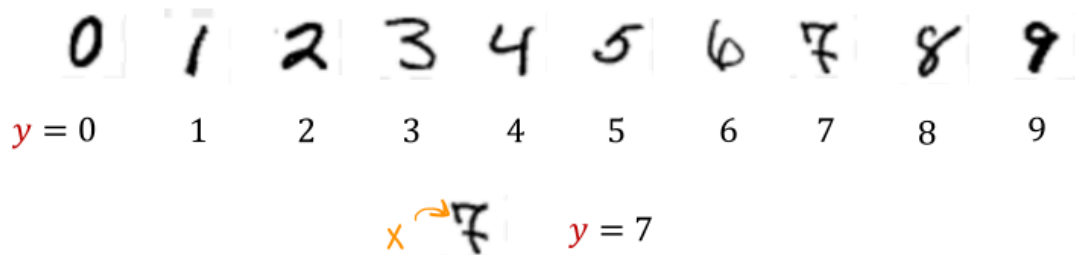
```
from tf.keras.layers import Dense
model = Sequential([
```

```
Dense(units=25, activation='relu'),
Dense(units=15, activation='relu'),
Dense(units=1, activation='sigmoid')
])
```

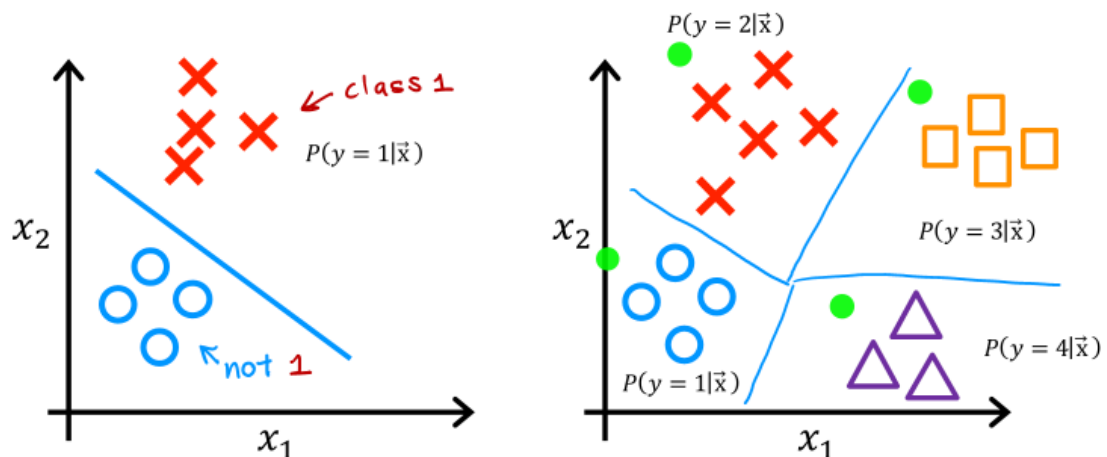
▼ Multiclass

- Bài toán Multiclass là khi bài toán phân loại mà target y có thể có nhiều hơn 2 giá trị.

MNIST example



Multiclass classification example



▼ Softmax

- Về bản chất softmax là một generalization cho logistic regression.
- Logistics regression được sử dụng khi y có 2 giá trị, và được tính như sau:

$$\begin{aligned}
 z &= \vec{w} \cdot \vec{x} + b \\
 \times a_1 &= g(z) = \frac{1}{1+e^{-z}} = P(y=1|\vec{x}) \quad 0.71 \\
 \bigcirc a_2 &= 1 - a_1 = P(y=0|\vec{x}) \quad 0.29
 \end{aligned}$$

Với softmax, chẳng hạn trong trường hợp có 4 output, ta có các giá trị z, được chuyển ở trên phần mũ của e, ta có: Và các vector w1, w2, w3, w4, b1, b1, b3, b4 là các param cho softmax.

$$\times z_1 = \vec{w}_1 \cdot \vec{x} + b_1$$

$$\bigcirc z_2 = \vec{w}_2 \cdot \vec{x} + b_2$$

$$\square z_3 = \vec{w}_3 \cdot \vec{x} + b_3$$

$$\triangle z_4 = \vec{w}_4 \cdot \vec{x} + b_4$$

Sử dụng các giá trị này, ta có probability rơi vào các trường hợp y=1,2,3,4 lần lượt là:

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y = 1 | \vec{x})$$

X O □ △

$$a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y = 2 | \vec{x})$$

$$a_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y = 3 | \vec{x})$$

$$a_4 = \frac{e^{z_4}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y = 4 | \vec{x})$$

→ Tựu chung lại trong trường hợp có N output:

Softmax regression
(N possible outputs) $y = 1, 2, 3, \dots, N$

$$z_j = \vec{w}_j \cdot \vec{x} + b_j \quad j = 1, \dots, N$$

parameters $w_1, w_2, \dots, w_N$
 $b_1, b_2, \dots, b_N$

$$a_j = \frac{e^{z_j}}{\sum_{k=1}^N e^{z_k}} = P(y = j | \vec{x})$$

note: $a_1 + a_2 + \dots + a_N = 1$

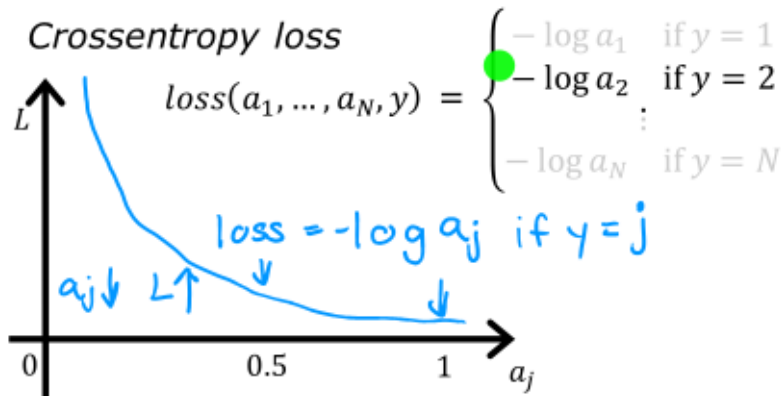
Hàm mất mát:

Softmax regression

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + \dots + e^{z_N}} = P(y = 1|\vec{x})$$

$$\vdots$$

$$a_N = \frac{e^{z_N}}{e^{z_1} + e^{z_2} + \dots + e^{z_N}} = P(y = N|\vec{x})$$



- Chẳng hạn như trong trường hợp ta có nhãn $y = 2$, thì Crossentropy loss của hàm lúc đó chỉ là $-\log a_2$.
 - Nhớ rằng, các giá trị a_N ở bên trên sẽ luôn luôn nằm trong khoảng từ 0 tới 1
- a_N chính là probability mà output của mô hình CHÍNH LÀ N , nên là nếu chẳng hạn như tính ra $a_N = 0.91$, thì loss sẽ rất thấp, và không âm do mình lấy dấu âm
- Nhớ rằng trong các trường hợp này, việc tính z chính là việc mình tính ra cái **raw, unnormalized log-probability output** (gọi là **logit**)
 - z nó chính là các con số thật, kiểu $-5.2, 12.4, \dots$
 - Sau đó nó được đưa qua e^z để đưa về số âm, trước khi chia cho tổng các e^z khác để đưa ra tỷ lệ a_N .

▼ Neural Network with Softmax output

Giờ khi mình muốn áp dụng vào một mô hình nhận biết con số từ 0 tới 9 chẳng hạn, thì ở output layer, mình sẽ đặt sao cho units = 10, và activation = 'softmax'. Cũng nhớ rằng trong các lớp ẩn ở bên trong, mình sẽ cho activation = 'relu'.

- Softmax activation có một điểm đặc biệt là để nhận được activation value ấy, mình sẽ phải dựa vào tất cả các giá trị z, thay vì chỉ là element-wise

MNIST with softmax

① specify the model

$f_{\vec{w},b}(\vec{x}) = ?$

② specify loss and cost

$L(f_{\vec{w},b}(\vec{x}), y)$

③ Train on data to minimize $J(\vec{w}, b)$

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
model = Sequential([
    Dense(units=25, activation='relu'),
    Dense(units=15, activation='relu'),
    Dense(units=10, activation='softmax')
])
from tensorflow.keras.losses import
    SparseCategoricalCrossentropy
model.compile(loss= SparseCategoricalCrossentropy() )
model.fit(X,Y,epochs=100)
```

- Hàm loss của mô hình này được gọi là SparseCategoricalCrossentropy.

▼ Improved implementation of softmax

Numerical Roundoff Errors

More numerically accurate implementation of logistic loss: $1 + \frac{1}{10,000}$ $1 - \frac{1}{10,000}$

Logistic regression:

$$\hat{a} = g(z) = \frac{1}{1 + e^{-z}}$$

Original loss

$$loss = -y \log(\hat{a}) - (1 - y) \log(1 - \hat{a})$$

More accurate loss (in code)

$$loss = -y \log\left(\frac{1}{1 + e^{-z}}\right) - (1 - y) \log\left(1 - \frac{1}{1 + e^{-z}}\right)$$

logit: z

```
model = Sequential([
    Dense(units=25, activation='relu'),
    Dense(units=15, activation='relu'),
    Dense(units=1, activation='sigmoid')
])
model.compile(loss=BinaryCrossEntropy())
model.compile(loss=BinaryCrossEntropy(from_logits=True))
```

→ Tầng cuối cùng của mô hình dùng `activation='sigmoid'` để tính ra xác suất $a = 1/(1+e^{-z})$. Giá trị a này bị máy tính giới hạn độ chính xác (ví dụ giữ lại 15 chữ số thập phân).

- Nếu mình sửa thành activation = 'linear' ở lớp cuối, ta sẽ ép mô hình xuất ra giá trị **Logit z** thô trực tiếp, chưa bị nén qua hàm Sigmoid để tránh bị mất độ chính xác.
- Sau đó bật `from_logits=True` khi compile, mình báo cho TensorFlow biết: "Đầu ra tôi đưa vào là một Logit z thô. Hãy tự gộp toán học Sigmoid + CrossEntropy lại với nhau để tính toán một cách chính xác nhất."
- TensorFlow biết rõ sử dụng hàm Sigmoid là **do tên gọi của hàm Loss mà mình khai báo trong lệnh** `model.compile()`
- Bản thân lớp đối tượng `BinaryCrossentropy` được TensorFlow lập trình cứng (hard-coded) để phục vụ cho **bài toán phân loại nhị phân (2 lớp)**. Về mặt toán học, phép toán phân loại nhị phân bắt buộc phải sử dụng hàm **Sigmoid** để nén giá trị logit thô về khoảng xác suất [0, 1].

Cũng tương tự khi nó sẽ tự động tính Softmax nếu mình viết hàm loss là `SparseCategoricalCrossentropy`.

- Tương tự như vậy trong softmax:

Softmax regression

$$(a_1, \dots, a_{10}) = g(z_1, \dots, z_{10})$$

$$\text{Loss} = L(\vec{a}, y) = \begin{cases} -\log a_1 & \text{if } y = 1 \\ \vdots \\ -\log a_{10} & \text{if } y = 10 \end{cases}$$

`model.compile()`

More Accurate

$$L(\vec{a}, y) = \begin{cases} -\log \frac{e^{z_1}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 1 \\ \vdots \\ -\log \frac{e^{z_{10}}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 10 \end{cases}$$

```
model = Sequential([
    Dense(units=25, activation='relu'),
    Dense(units=15, activation='relu'),
    Dense(units=10, activation='linear')
])
```

```
model.compile(loss=SparseCrossEntropy(from_logits=True))
```

→ Do khi mình sử dụng activation = 'linear', mô hình giờ sẽ tính ra z_1 đến z_{10} , chứ không phải là a_1 đến a_{10} , nên mình phải lưu các biến logits riêng xong sau đó predict bằng các sử dụng hàm softmax ngoài, được truyền vào biến logits đấy.

MNIST (more numerically accurate)

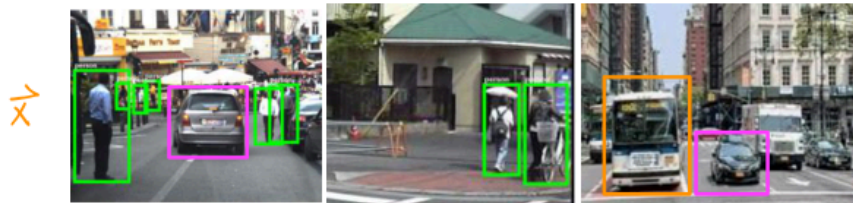
```
model    import tensorflow as tf
         from tensorflow.keras import Sequential
         from tensorflow.keras.layers import Dense
         model = Sequential([
             Dense(units=25, activation='relu')
             Dense(units=15, activation='relu')
             Dense(units=10, activation='linear') ])
loss     from tensorflow.keras.losses import
         SparseCategoricalCrossentropy

         model.compile(..., loss=SparseCategoricalCrossentropy(from_logits=True) )
fit      model.fit(X,Y, epochs=100)
predict  logits = model(X) ← not  $a_1 \dots a_{10}$ 
         f_x = tf.nn.softmax(logits)      is  $z_1 \dots z_{10}$ 
```

Hàm thủ công là `tf.nn.softmax()`

▼ Classification with multiple outputs

Multi-label Classification



Is there a car?

yes

Is there a bus?

no

Is there a pedestrian?

yes

$$y = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

no

no

yes

$$y = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

yes

yes

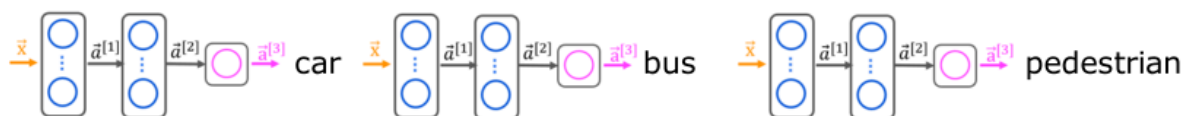
no

$$y = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$$

- Về bản chất nó như kiểu mình có nhiều bài toán phân loại, chồng lên nhau. Chẳng hạn trong trường hợp này sẽ có thể có bài toán phân loại về car, bus, và pedestrian riêng lẻ.

Mình vẫn có thể tạo ra một mô hình nơi mà cái output layer sẽ đưa ra 3 nơ ron, mỗi nơ ron đại diện cho từng bài toán một:

Multiple classes



Alternatively, train one neural network with three outputs



▼ Advanced Optimization

Bài này nói về một thuật toán mà sẽ giúp huấn luyện nhanh hơn Gradient Descent, là Adam

GD:

$$w_j = w_j - \alpha \frac{\partial J}{\partial w_j}$$

- Ở trong Gradient Descent nó dựa vào một global learning rate duy nhất, cho tất cả các tham số.
 - **Khi alpha quá nhỏ:** Thuật toán thực hiện những bước đi rất ngắn và lặp đi lặp lại theo cùng một hướng về phía điểm cực tiểu. Quá trình này không hiệu quả và mất quá nhiều thời gian để hội tụ.
 - **Khi alpha quá lớn:** Thuật toán đi vượt quá (overshoot) điểm cực tiểu, khiến nó dao động và bật qua bật lại trên đồ thị đường đồng mức (hàm chi phí) thay vì đi theo một đường cong mượt mà hướng xuống đáy.
- Adam giải quyết các hạn chế của Gradient Descent bằng cách **tự động điều chỉnh tốc độ học** dựa trên diễn biến của quá trình giảm dốc.
 - **Nếu một tham số liên tục di chuyển theo cùng một hướng:** Adam tự động *tăng* tốc độ học cho riêng tham số đó để nó đạt đến cực tiểu nhanh hơn.
 - **Nếu một tham số liên tục dao động bật qua bật lại:** Adam tự động *giảm* tốc độ học cho tham số đó để ngừng việc nảy lên nảy xuống và làm mượt đường đi.

→ Nó gán một tốc độ học riêng biệt cho *từng tham số một* trong mô hình. Chẳng hạn như nếu có 10 tham số w , 1 tham số b , thì sẽ có 11 learning rates.

- Adam cần một **tốc độ học khởi tạo mặc định (đề là 0.001)**

```
model = Sequential([
    tf.keras.layers.Dense(units=25, activation='sigmoid'),
    tf.keras.layers.Dense(units=15, activation='sigmoid'),
    tf.keras.layers.Dense(units=10, activation='linear')
])
```

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_
rate=1e-3),
               loss=tf.keras.losses.SparseCategoricalCrosse
entropy(from_logits=True))

model.fit(X,Y,epochs=100)
```

▼ Additional Layer Types

- Đến giờ mình mới chỉ sử dụng Dense layer, nhưng ta có một kiểu layer khác nổi bật nhất là Convolution Layer, tạo thành một Convolution Neural Networks (CNN)

1. Dense layer:

- **Đặc điểm:** Mỗi nơ-ron trong một lớp ẩn sẽ nhận đầu vào là **tất cả** các giá trị kích hoạt (activations) từ lớp ngay trước nó.
- *Hạn chế:* Với dữ liệu lớn như hình ảnh, việc mỗi nơ-ron phải "nhìn" vào tất cả pixel sẽ tạo ra khối lượng tính toán khổng lồ và dễ gây nhiễu.

2. Convolutional Layer (lớp tích chập)

- Thay vì nhìn vào toàn bộ đầu vào, mỗi nơ-ron chỉ "nhìn" vào một **vùng giới hạn** của dữ liệu.

VD:

- Nơ-ron thứ nhất không xử lý toàn bộ bức ảnh, nó chỉ phân tích một vùng pixel hình chữ nhật nhỏ ở góc.
- Nơ-ron thứ hai sẽ phân tích một vùng pixel liền kề khác, cứ thế quét qua toàn bộ bức ảnh.
- Các phần mà từng nơ-ron nhìn vào hoàn toàn có thể trùng nhau.

→ **Tăng tốc độ tính toán:** Do nơ-ron chỉ xử lý một lượng nhỏ dữ liệu đầu vào.

→ **Cần ít dữ liệu huấn luyện hơn:** Tối ưu hóa việc trích xuất đặc trưng (feature extraction).

→ **Giảm thiểu Overfitting (Học vẹt):** Mô hình khái quát hóa dữ liệu tốt hơn.

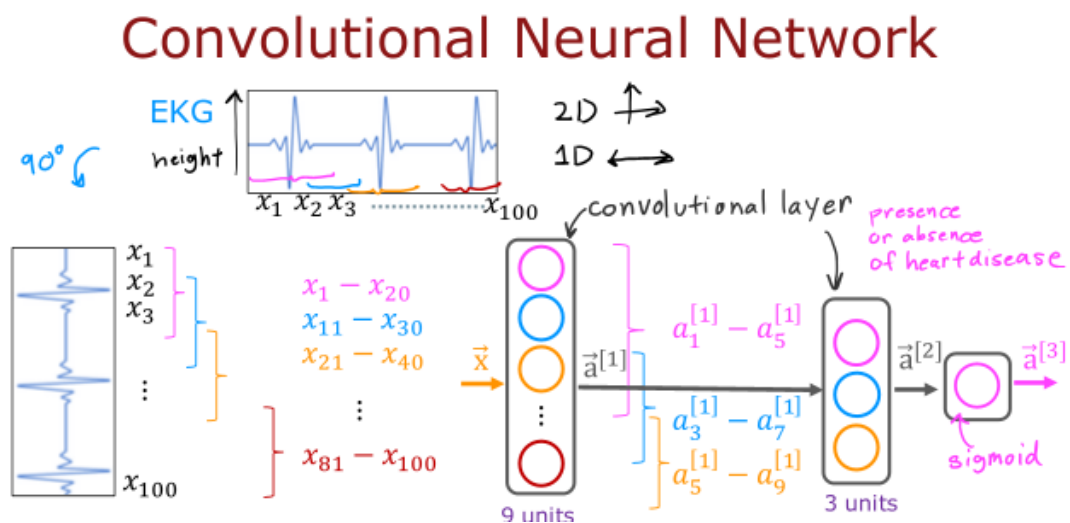
▼ Ví dụ về EKG

Phân loại EKG bằng CNN 1D

Thay vì ảnh 2D, chúng ta xét dữ liệu 1D theo chuỗi thời gian như tín hiệu nhịp tim (EKG/ECG) để chẩn đoán bệnh lý tim mạch.

Giả sử tín hiệu EKG được ghi lại thành 100 điểm dữ liệu theo thời gian (từ x_1 đến x_{100}):

- **Lớp ẩn thứ 1 (Convolutional Layer):** Thay vì nhận cả 100 điểm đầu vào.
 - Unit 1 chỉ xử lý một cửa sổ nhỏ: x_1 đến x_{20}
 - Unit 2 xử lý cửa sổ tiếp theo: x_{11} đến x_{30}
 - ... Unit cuối cùng xử lý: x_{81} đến x_{100}
- **Lớp ẩn thứ 2 (Convolutional Layer):** Nhận đầu ra của lớp 1 (gọi là tập hợp các giá trị activation " a "). Giả sử lớp 1 có 9 units.
 - Unit 1 của lớp 2 chỉ nhìn vào 5 activations đầu tiên: a_1 đến a_5
 - Unit 2 nhìn vào: a_3 đến a_7
 - Unit 3 nhìn vào: a_5 đến a_9
- **Lớp đầu ra (Output Layer):** Sử dụng một lớp Sigmoid nhận dữ liệu từ các unit của lớp 2 để đưa ra phân loại nhị phân cuối cùng (Ví dụ: dự đoán bằng 1 là có bệnh, bằng 0 là không có bệnh).



▼ ADAM - Adaptive Movement Estimation

- ADAM là một trong những thuật toán phổ biến và được áp dụng nhiều nhất trong học máy.
- ADAM trả lời cho câu hỏi: làm sao để di chuyển đến cực tiểu trên đồ thị của hàm mất mát một cách nhanh chóng, ổn định và không bị mắc kẹt (ở các cực tiểu local). ADAM áp dụng 2 con số:
 - Quán tính (Momentum): Nếu đang đi đúng hướng, hãy lấy đà để đi nhanh hơn và vượt qua các gờ nhỏ.
 - Thích ứng (RMSprop): Nếu đường đi quá gập ghềnh hoặc dốc đứng, hãy tự động phanh lại.

Điều đầu tiên ta làm trong một thuật toán tối ưu tham số là tính đạo hàm, nên ta có:

$$g_t = \nabla_{\theta} f(\theta_{t-1})$$

▼ Trong đó:

1. Vector gradient (độ dốc):

$$g_t$$

2. Toán tử gradient:

$$\nabla_{\theta}$$

3. Hàm mất mát, áp dụng với tham số của lần trước:

$$f(\theta_{t-1})$$

Lưu ý: Bản chất Theta ở đây là một vector tham số:

$$\theta = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix}$$

Do vậy chính g_t cũng là một vector gradient, với mỗi element bên trong vector là sự đạo hàm của hàm cost theo từng tham số tương ứng :

$$g_t = \nabla_{\theta} f(\theta) = \begin{bmatrix} \frac{\partial f}{\partial w_1} \\ \frac{\partial f}{\partial w_2} \\ \frac{\partial f}{\partial b} \end{bmatrix}$$

Sau khi tính gradient xong, ta sẽ lấy và tính momen (gọi là momen bậc 1) theo công thức:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

▼ Trong đó:

1. m_t là đà di chuyển tại thời điểm t
2. Beta 1 là hệ số giảm chấn bậc 1 (thường là 0.9)
3. m_{t-1} là đà cũ của lần trước
4. g_t là vector gradient

Ta nhận thấy với hệ số giảm chấn bậc 1 là 0.9, công thức sẽ giữ nguyên 90% đà từ quá khứ, cộng với 10% độ dốc (gradient) của hiện tại. Quán tính giúp mô hình di chuyển mượt mà hơn, bỏ qua các điểm nhiễu.

Nhớ rằng do g_t chính là đạo hàm, nên nó vẫn có thể mang dấu âm, điều này giúp công thức sẽ mang đặc tính tăng giảm của đà.

Tính xong momentum bậc 1, ta sẽ tính nốt con số còn lại của ADAM là momen bậc 2, với công thức:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

▼ Trong đó:

1. Momen bậc 2 (độ biến động): v_t
2. Beta 2 là hệ số giảm chấn bậc 2 (thường là 0.999)
3. v_{t-1} là momen bậc 2 cũ từ trước đó

Công thức sử dụng bình phương của đạo hàm, nên thực chất, nó đang tính ra độ lớn của độ dốc ở vị trí hiện tại là mạnh hay yếu (bởi vì ở công thức cuối cùng sẽ lấy căn), bất kể là đang đi lên hay đi xuống.

vt lưu giữ 99,9% độ lớn trong quá khứ và 0.1% độ lớn hiện tại, tuy vậy, momen bậc 2 vẫn có thể tăng hoặc giảm tương tự như bậc 1, phụ thuộc vào đạo hàm ở vị trí hiện tại.

Cụ thể, vt sẽ giảm khi bình phương đạo hàm hiện tại nhỏ hơn momen bậc 2 của vị trí trước đó ($gt^2 < vt-1$).

Có một điều mình có thể nhận ra là ở trong giai đoạn đầu khi thuật toán mới khởi động, thì các giá trị của lần chạy trước là bằng 0 ($mt-1 = vt-1 = 0$), thì sẽ mất rất lâu để tính được momen bậc 1 (chỉ lấy 10% giá trị mới), và momen bậc 2 (chỉ lấy 0.1% giá trị mới).

Do vậy, để đảm bảo công bằng cho mỗi lần chạy đầu, ta sẽ phải hiệu chỉnh sai số trước khi cập nhật trọng số thật sự, với các công thức:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$
$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Trong đó, các hệ số giảm chấn sẽ được lũy thừa lên mũ số t (số vòng lặp hiện tại). Khi t càng lớn (khi huấn luyện lâu), thì β^t sẽ tiến dần về 0, lúc này mẫu số tiến về 1, nghĩa là sự hiệu chỉnh này sẽ tự động mất đi khi không còn cần thiết nữa.

Cuối cùng mình sẽ cập nhật vector tham số với công thức:

$$\theta_t = \theta_{t-1} - \frac{\alpha \cdot \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

▼ Trong đó:

1. Alpha là learning rate (thường là 0.001)
2. Epsilon là một hằng số cực kỳ nhỏ (thường là 10^{-8})

Căn bậc 2 của vt như kiểu một "chiếc phanh" cho từng tham số. Nếu một tham số có gradient biến động rất mạnh (giá trị này lớn), learning sẽ bị chia cho một số lớn, làm bước nhảy ngắn lại. Ngược lại, nếu tham số ít thay đổi, bước nhảy sẽ được tăng lên.

Epsilon chỉ là một hằng số cực kỳ nhỏ (10^{-8}) để cộng vào mẫu số để ngăn cho lỗi toán học chia cho 0.

▼ Derivatives

Về bản chất đạo hàm chính là dùng để đo lường sự thay đổi tương đương giữa một biến bên trong hàm, và giá trị của hàm đó. Khái niệm này còn được gọi là "tỉ lệ thay đổi" (rate of change), hoặc "độ nhạy" (sensitivity).

Như kiểu "Nếu tôi nhích biến x lên một lượng cực kỳ nhỏ, thì giá trị y sẽ tăng hay giảm gấp bao nhiêu lần?"

- Đối với trong học máy, giá trị của Loss Function, là thước đo xem mô hình đang dự đoán tệ đến mức nào. Mục tiêu của mình là phải ép cái sai số này xuống càng thấp càng tốt.
- Đối với các biến bên trong cái hàm này, thì đó là hàng nghìn, hàng triệu trọng số (mặc khác là các tham số của vector θ , của mạng nơ ron.
- Vector gradient gt khi đó, là tập hơn tất cả các đạo hàm của mọi ẩn số, và nó như là một dấu hiệu để mình thấy được "sensitivity" của từng tham số một, nó có ảnh hưởng như nào tới giá trị của Loss Function.